

# Markov Decision Processes and RL Foundations

Mathematics of Deep Learning — Chapter 12

---

Yin Yang

Foundation Books Project

# Roadmap

From Prediction to Sequential Decisions

States, Actions, Rewards, Trajectories

Markov Decision Processes

Policies as Decision Rules

Returns, Discounting, Objectives

Value Functions

Bellman Expectation Equations

Optimality and Dynamic Programming

Monte Carlo and TD Targets

Q-Learning and DQN

Policy Gradients

Advantages and Actor-Critic

Proximal Policy Optimization

Why Deep RL Is Hard

Summary

# From Prediction to Sequential Decisions

---

# What Changes vs. Supervised Learning

Supervised learning: dataset  $\mathcal{D} = \{(x_i, y_i)\}$  is fixed.

$$\hat{R}(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(f_{\theta}(x_i), y_i).$$

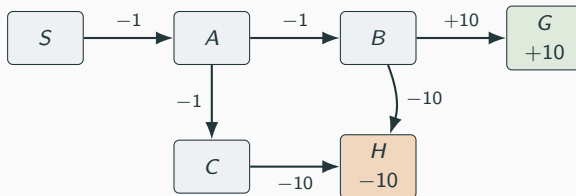
Reinforcement learning: data is a stream

$$S_0, A_0, R_1, S_1, A_1, R_2, \dots$$

- The learner's choices change which states it sees next.
- Useful feedback can arrive several steps after the action.
- Total reward, exploration, and distribution shift matter — not only one-step loss.

In RL, the model helps create the data it trains on.

## Running Example: Delivery Grid



- Goal  $G$ :  $+10$ . Hazard  $H$ :  $-10$ . Step cost:  $-1$ .
- One episode:  $S$ , right,  $-1$ ,  $A$ , right,  $-1$ ,  $B$ , right,  $+10$ ,  $G$ .
- The same first move can be good or bad depending on what follows.

Small grid, but the symbols scale to games, robots, and LLM policies.

## **States, Actions, Rewards, Trajectories**

---

## One Transition: $S_t, A_t, R_{t+1}, S_{t+1}$

At time  $t$ :

$$S_t \in \mathcal{S}, \quad A_t \in \mathcal{A}(S_t), \quad R_{t+1} \in \mathbb{R}, \quad S_{t+1} \in \mathcal{S}.$$

| Symbol    | Role                           | Timing                 |
|-----------|--------------------------------|------------------------|
| $S_t$     | current state for the decision | observed before acting |
| $A_t$     | action chosen by the agent     | chosen in $S_t$        |
| $R_{t+1}$ | scalar feedback                | received after $A_t$   |
| $S_{t+1}$ | next state                     | observed with reward   |

A trajectory:  $\tau = (s_0, a_0, r_1, s_1, a_1, r_2, \dots, s_T)$ . An episode ends at a terminal state ( $G$  or  $H$ ).

One transition groups current state, action, next reward, next state.

# Markov Decision Processes

---



# The MDP Tuple

A finite discounted MDP is

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma).$$

- $P(s' | s, a) = \mathbb{P}(S_{t+1} = s' | S_t = s, A_t = a)$  is a row-stochastic kernel:  $P \geq 0$ ,  $\sum_{s'} P(s' | s, a) = 1$ .
- $r(s, a, s')$  is the expected reward of that transition.
- $\gamma \in [0, 1)$  is the discount factor.

**Markov property:**

$$\mathbb{P}(S_{t+1} = s' | S_0, A_0, \dots, S_t = s, A_t = a) = P(s' | s, a).$$

The current state-action pair carries all the information needed for the next-state distribution.

A useful state contains what is needed for the next transition.

# Stochastic Transitions: A Slippery Right Move

Action right from  $A$ :

$$P(B \mid A, \text{right}) = 0.9, \quad P(C \mid A, \text{right}) = 0.1.$$

Both have reward  $-1$ . The expected next-state value, for any  $V$ , is

$$\sum_{s'} P(s' \mid A, \text{right}) V(s') = 0.9 V(B) + 0.1 V(C).$$

- Environment randomness (kernel  $P$ ) is separate from the agent's randomness (policy).
- The Markov sum collapses any history into the current pair  $(s, a)$ .

Conditional expectations become finite weighted sums over  $s'$ .

## Policies as Decision Rules

---

# Deterministic and Stochastic Policies

A *deterministic* policy is  $\mu : \mathcal{S} \rightarrow \mathcal{A}$ . A *stochastic* policy is

$$\pi(a \mid s) = \mathbb{P}(A_t = a \mid S_t = s), \quad \sum_{a \in \mathcal{A}} \pi(a \mid s) = 1.$$

Deep policy network with discrete actions:

$$\pi_{\theta}(a \mid s) = \text{softmax}(z_{\theta}(s))_a = \frac{\exp(z_{\theta,a}(s))}{\sum_b \exp(z_{\theta,b}(s))}.$$

Looks like a classifier — but there is no fixed correct-action label. Actions are judged by later reward.

Trajectory probability:

$$p_{\pi}(s_0, a_0, \dots, s_T) = \rho_0(s_0) \prod_{t=0}^{T-1} \pi(a_t \mid s_t) P(s_{t+1} \mid s_t, a_t).$$

A policy network predicts action probabilities, not class labels.

# Returns, Discounting, Objectives

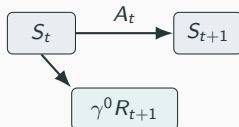
---

## Discounted Return (Animated)

$$S_t$$

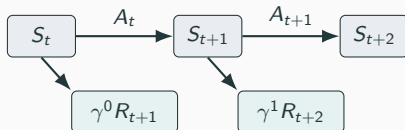
(1) Start from  $S_t$ .

## Discounted Return (Animated)



(1) Start from  $S_t$ . (2) First reward arrives undiscounted.

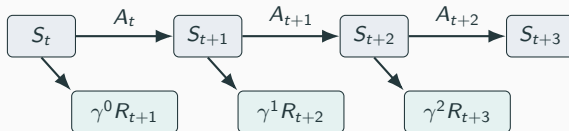
## Discounted Return (Animated)



(1) Start from  $S_t$ . (2) First reward arrives undiscounted. (3) Future rewards get factor  $\gamma$ .

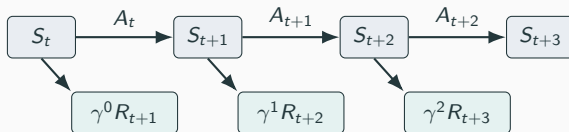


## Discounted Return (Animated)



**(1)** Start from  $S_t$ . **(2)** First reward arrives undiscounted. **(3)** Future rewards get factor  $\gamma$ . **(4)** Even later rewards get  $\gamma^2, \gamma^3, \dots$

## Discounted Return (Animated)



$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

**(1)** Start from  $S_t$ . **(2)** First reward arrives undiscounted. **(3)** Future rewards get factor  $\gamma$ . **(4)** Even later rewards get  $\gamma^2, \gamma^3, \dots$  **(5)** Sum is one scalar return.

Discounting turns a delayed reward stream into one training signal.

## Discounting: Numbers and a Bound

Path  $S \rightarrow A \rightarrow B \rightarrow G$  has rewards  $-1, -1, 10$ . With  $\gamma = 0.9$ ,

$$G_0 = -1 + 0.9(-1) + 0.9^2(10) = 6.2.$$

Bound on infinite-horizon return: if  $|R_{t+1}| \leq R_{\max}$  and  $0 \leq \gamma < 1$ ,

$$|G_t| \leq \sum_{k=0}^{\infty} \gamma^k R_{\max} = \frac{R_{\max}}{1 - \gamma}.$$

Policy objective:

$$J(\pi) = \mathbb{E}_{\rho_0, \pi, P}[G_0].$$

- $\gamma = 0$ : only the next reward counts.
- $\gamma$  near 1: long horizon, but a slower numerical signal.

Choose a policy with large  $\mathbb{E}[G_0]$ , not small per-example loss.

# Value Functions

---

# State-Value and Action-Value

## State value

$V^\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$  — expected return from  $s$  under policy  $\pi$ .

## Action value

$Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$  — expected return after forcing the first action to be  $a$ .

Example. From  $A$ , policy picks right 0.7, down 0.3. Right-return 8, down-return  $-10$ :

$$V^\pi(A) = 0.7(8) + 0.3(-10) = 2.6.$$

Deep RL approximates these with neural nets:

$$V_\phi(s) \approx V^\pi(s), \quad Q_\phi(s, a) \approx Q^\pi(s, a).$$

Control needs  $Q$  to compare actions, not just  $V$  for one state.

# Bellman Expectation Equations

---

# Bellman Expectation: One-Step Self-Consistency

For any fixed policy  $\pi$ ,

$$V^\pi(s) = \sum_a \pi(a | s) \sum_{s'} P(s' | s, a) (r(s, a, s') + \gamma V^\pi(s')).$$

- Decompose:  $G_t = R_{t+1} + \gamma G_{t+1}$ .
- Average over actions sampled from  $\pi$  and next states from  $P$ .

Action-value form:

$$Q^\pi(s, a) = \sum_{s'} P(s' | s, a) (r(s, a, s') + \gamma \sum_{a'} \pi(a' | s') Q^\pi(s', a')).$$

Numerical backup with  $V^\pi(B) = 8$ ,  $V^\pi(C) = -10$ ,  $\gamma = 0.9$ :

$$V^\pi(A) = 0.7(-1 + 0.9 \cdot 8) + 0.3(-1 + 0.9(-10)) = 1.34.$$

A Bellman backup is the return definition plus one-step conditioning.

# Optimality and Dynamic Programming

---



# Bellman Optimality and Value Iteration

Optimal values:  $V^*(s) = \max_{\pi} V^{\pi}(s)$ ,  $Q^*(s, a) = \max_{\pi} Q^{\pi}(s, a)$ .

$$V^*(s) = \max_a \sum_{s'} P(s' | s, a) (r(s, a, s') + \gamma V^*(s')).$$

Greedy action:  $\arg \max_a Q^*(s, a)$ .

**Bellman optimality operator  $\mathcal{T}$ :**

$$(\mathcal{T}V)(s) = \max_a \sum_{s'} P(s' | s, a) (r(s, a, s') + \gamma V(s')).$$

$\mathcal{T}$  is a  $\gamma$ -contraction in  $\|\cdot\|_{\infty}$ :

$$\|\mathcal{T}V - \mathcal{T}W\|_{\infty} \leq \gamma \|V - W\|_{\infty}.$$

Value iteration:  $V_{k+1} = \mathcal{T}V_k$  converges to  $V^*$ . **Maximization (control)** replaces averaging (policy evaluation).

## Monte Carlo and TD Targets

---

# Monte Carlo vs. TD Targets

Sample-based methods replace expectations by targets built from data.

**Monte Carlo:** wait for the rollout return,  $\hat{V}(S_t) \leftarrow G_t$ .

**TD(0):** bootstrap from current estimate,

$$Y_t = R_{t+1} + \gamma V_\phi(S_{t+1}), \quad \delta_t = Y_t - V_\phi(S_t).$$

| Target      | Formula                            | Available after | Bootstraps? |
|-------------|------------------------------------|-----------------|-------------|
| Monte Carlo | $G_t = \sum_k \gamma^k R_{t+k+1}$  | episode suffix  | no          |
| TD(0)       | $R_{t+1} + \gamma V_\phi(S_{t+1})$ | one transition  | yes         |

With  $R_{t+1} = -1$ ,  $\gamma = 0.9$ ,  $V_\phi(S_t) = 2.0$ ,  $V_\phi(S_{t+1}) = 4.0$ :  $Y_t = 2.6$ ,  $\delta_t = 0.6$ . Bootstrapping learns from every transition — at the cost of a moving target.

## Q-Learning and DQN

---

# Q-Learning Target and Update

Q-learning target:

$$Y_t = R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a').$$

Tabular update:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha(Y_t - Q(S_t, A_t)).$$

Off-policy: data may be exploratory; targets still use  $\max_{a'}$ .

Numerical step:  $Q(S_t, A_t) = 2.0$ ,  $R_{t+1} = -1$ ,  $\gamma = 0.9$ ,  $\max_{a'} Q(S_{t+1}, a') = 5.0$ ,  $\alpha = 0.1$ :

$$Y_t = -1 + 0.9(5.0) = 3.5, \quad Q(S_t, A_t) \leftarrow 2.0 + 0.1(1.5) = 2.15.$$

Q-learning learns greedy action values from possibly exploratory data.

# Deep Q-Networks: Stabilizing Bootstrapped Regression

transition  
stream

$$L(\theta) = \left( R_{t+1} + \gamma \max_{a'} Q_{\bar{\theta}}(S_{t+1}, a') - Q_{\theta}(S_t, A_t) \right)^2.$$

(1) Stream of  $(s, a, r, s')$  tuples.

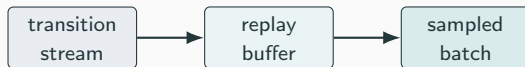
# Deep Q-Networks: Stabilizing Bootstrapped Regression



$$L(\theta) = \left( R_{t+1} + \gamma \max_{a'} Q_{\bar{\theta}}(S_{t+1}, a') - Q_{\theta}(S_t, A_t) \right)^2.$$

**(1)** Stream of  $(s, a, r, s')$  tuples. **(2)** Replay buffer breaks correlation.

# Deep Q-Networks: Stabilizing Bootstrapped Regression

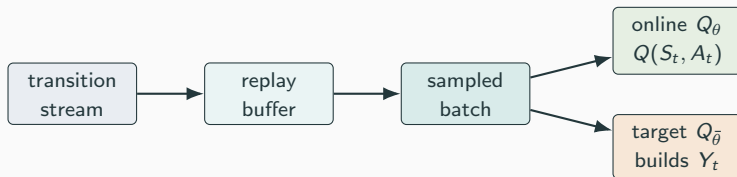


$$L(\theta) = \left( R_{t+1} + \gamma \max_{a'} Q_{\bar{\theta}}(S_{t+1}, a') - Q_{\theta}(S_t, A_t) \right)^2.$$

**(1)** Stream of  $(s, a, r, s')$  tuples. **(2)** Replay buffer breaks correlation. **(3)** Sample uniform minibatches.



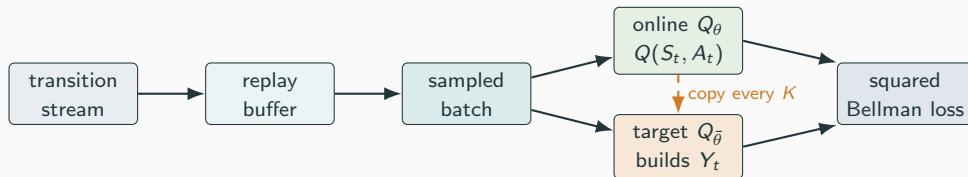
# Deep Q-Networks: Stabilizing Bootstrapped Regression



$$L(\theta) = \left( R_{t+1} + \gamma \max_{a'} Q_{\bar{\theta}}(S_{t+1}, a') - Q_\theta(S_t, A_t) \right)^2.$$

- (1)** Stream of  $(s, a, r, s')$  tuples. **(2)** Replay buffer breaks correlation. **(3)** Sample uniform minibatches. **(4)** Online net trained; target net frozen between syncs.

# Deep Q-Networks: Stabilizing Bootstrapped Regression



$$L(\theta) = (R_{t+1} + \gamma \max_{a'} Q_{\bar{\theta}}(S_{t+1}, a') - Q_\theta(S_t, A_t))^2.$$

**(1)** Stream of  $(s, a, r, s')$  tuples. **(2)** Replay buffer breaks correlation. **(3)** Sample uniform minibatches. **(4)** Online net trained; target net frozen between syncs. **(5)** Squared Bellman loss; target updates slow the moving target.

Replay + target network turn bootstrap regression into stable training.

# Policy Gradients

---

## REINFORCE: The Score-Function Identity

Objective:  $J(\theta) = \mathbb{E}_{\tau \sim p_\theta}[G_0]$ . Log-derivative trick:

$$\nabla_\theta p_\theta(\tau) = p_\theta(\tau) \nabla_\theta \log p_\theta(\tau).$$

Only policy factors depend on  $\theta$ :

$$\nabla_\theta \log p_\theta(\tau) = \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(A_t | S_t).$$

Cancelling past rewards (score has zero conditional mean) gives

$$\nabla_\theta J(\theta) = \mathbb{E} \left[ \sum_{t=0}^{T-1} \gamma^t G_t \nabla_\theta \log \pi_\theta(A_t | S_t) \right].$$

Direction:  $\nabla_\theta \log \pi_\theta(A_t | S_t)$  makes the sampled action more likely; multiply by  $G_t$  to weight by how good that future was.

Make sampled actions more likely when the return that follows is good.

## **Advantages and Actor-Critic**

---

# Baselines and the Advantage Function (Animated)

state  $s$

Any state-only baseline  $b(s)$  leaves the expected gradient unchanged:

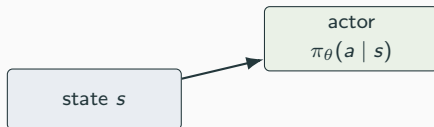
$$\mathbb{E}_{A \sim \pi_\theta} [b(s) \nabla_\theta \log \pi_\theta(A | s)] = 0.$$

Choose  $b(s) = V^\pi(s)$  to get the *advantage*:

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s).$$

**(1)** Same state feeds both heads.

# Baselines and the Advantage Function (Animated)



Any state-only baseline  $b(s)$  leaves the expected gradient unchanged:

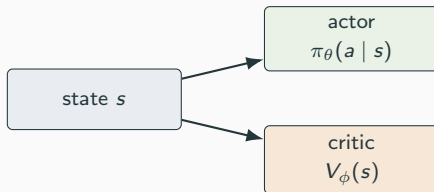
$$\mathbb{E}_{A \sim \pi_{\theta}} [b(s) \nabla_{\theta} \log \pi_{\theta}(A | s)] = 0.$$

Choose  $b(s) = V^{\pi}(s)$  to get the *advantage*:

$$A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s).$$

**(1)** Same state feeds both heads. **(2)** Actor produces  $\pi_{\theta}(a | s)$ .

# Baselines and the Advantage Function (Animated)



Any state-only baseline  $b(s)$  leaves the expected gradient unchanged:

$$\mathbb{E}_{A \sim \pi_{\theta}} [b(s) \nabla_{\theta} \log \pi_{\theta}(A | s)] = 0.$$

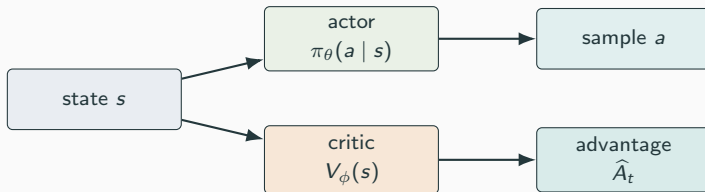
Choose  $b(s) = V^{\pi}(s)$  to get the *advantage*:

$$A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s).$$

**(1)** Same state feeds both heads. **(2)** Actor produces  $\pi_{\theta}(a | s)$ . **(3)** Critic estimates  $V_{\phi}(s)$ .



# Baselines and the Advantage Function (Animated)



Any state-only baseline  $b(s)$  leaves the expected gradient unchanged:

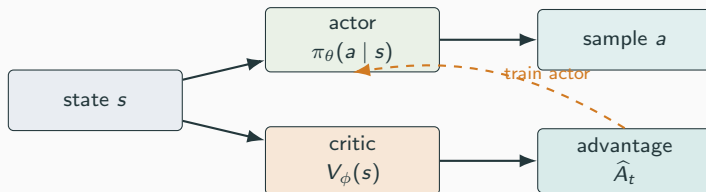
$$\mathbb{E}_{A \sim \pi_{\theta}} [b(s) \nabla_{\theta} \log \pi_{\theta}(A | s)] = 0.$$

Choose  $b(s) = V^{\pi}(s)$  to get the *advantage*:

$$A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s).$$

- (1)** Same state feeds both heads. **(2)** Actor produces  $\pi_{\theta}(a | s)$ . **(3)** Critic estimates  $V_{\phi}(s)$ .  
**(4)** Sample action; form  $\hat{A}_t$ .

# Baselines and the Advantage Function (Animated)



Any state-only baseline  $b(s)$  leaves the expected gradient unchanged:

$$\mathbb{E}_{A \sim \pi_{\theta}} [b(s) \nabla_{\theta} \log \pi_{\theta}(A | s)] = 0.$$

Choose  $b(s) = V^{\pi}(s)$  to get the *advantage*:

$$A^{\pi}(s, a) = Q^{\pi}(s, a) - V^{\pi}(s).$$

- (1)** Same state feeds both heads. **(2)** Actor produces  $\pi_{\theta}(a | s)$ . **(3)** Critic estimates  $V_{\phi}(s)$ . **(4)** Sample action; form  $\hat{A}_t$ . **(5)** Advantage trains the actor.

Advantage = action quality relative to the state.

# Performance Difference Lemma

Discounted state-occupancy under  $\pi'$ :

$$d_{\pi'}(s) = (1 - \gamma) \sum_{t=0}^{\infty} \gamma^t \mathbb{P}_{\pi'}(S_t = s).$$

## Performance difference

$$J(\pi') - J(\pi) = \frac{1}{1 - \gamma} \mathbb{E}_{s \sim d_{\pi'}, a \sim \pi'(\cdot|s)}[A^{\pi}(s, a)].$$

- Improvement is driven by positive advantages *under the old value function*, averaged over states the *new* policy visits.
- This identity is the starting point for trust-region and proximal policy methods.

Improvement = positive advantages on visitation of the new policy.

# Proximal Policy Optimization

---

# PPO Probability Ratio and Clipped Surrogate (Animated)

Probability ratio for a sampled action:

$$\rho_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}.$$



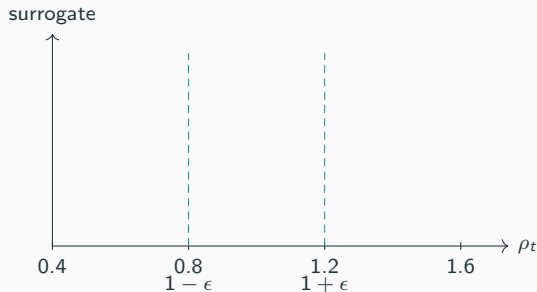
$$\ell_t^{\text{PPO}}(\theta) = \min\{\rho_t(\theta)\hat{A}_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t\}.$$

(1) Axes: ratio  $\rho_t$  vs. surrogate value.

# PPO Probability Ratio and Clipped Surrogate (Animated)

Probability ratio for a sampled action:

$$\rho_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}.$$



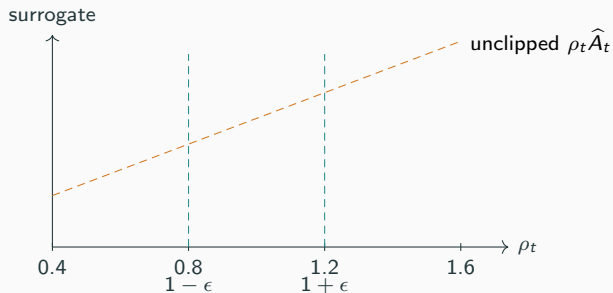
$$\ell_t^{\text{PPO}}(\theta) = \min\{\rho_t(\theta)\hat{A}_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t\}.$$

**(1)** Axes: ratio  $\rho_t$  vs. surrogate value. **(2)** Clip band  $[1 - \epsilon, 1 + \epsilon]$ .

# PPO Probability Ratio and Clipped Surrogate (Animated)

Probability ratio for a sampled action:

$$\rho_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}.$$



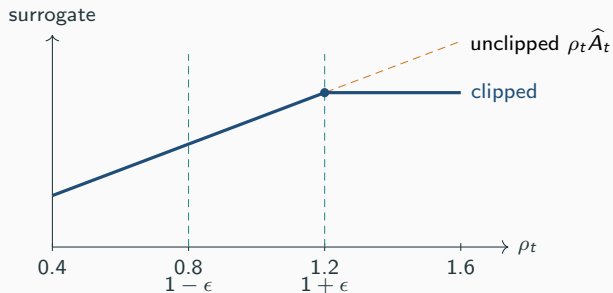
$$\ell_t^{\text{PPO}}(\theta) = \min\{\rho_t(\theta)\hat{A}_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t\}.$$

**(1)** Axes: ratio  $\rho_t$  vs. surrogate value. **(2)** Clip band  $[1 - \epsilon, 1 + \epsilon]$ . **(3)** Without clip: linear in  $\rho_t$ .

# PPO Probability Ratio and Clipped Surrogate (Animated)

Probability ratio for a sampled action:

$$\rho_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}.$$



$$\ell_t^{\text{PPO}}(\theta) = \min\{\rho_t(\theta)\hat{A}_t, \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t\}.$$

**(1)** Axes: ratio  $\rho_t$  vs. surrogate value. **(2)** Clip band  $[1 - \epsilon, 1 + \epsilon]$ . **(3)** Without clip: linear in  $\rho_t$ . **(4)** PPO: stop rewarding once outside the band.



# PPO Clipping: Worked Examples

Take  $\epsilon = 0.2$ .

**Positive advantage.**  $\hat{A}_t = 3, \rho_t = 1.5$ :

$$\text{unclipped} = 4.5, \quad \text{clipped} = 1.2(3) = 3.6, \quad \text{min} = 3.6.$$

**Negative advantage.**  $\hat{A}_t = -2, \rho_t = 0.6$ :

$$\text{unclipped} = -1.2, \quad \text{clipped} = 0.8(-2) = -1.6, \quad \text{min} = -1.6.$$

- For  $\hat{A}_t > 0$ : surrogate capped at  $(1 + \epsilon)\hat{A}_t$ .
- For  $\hat{A}_t < 0$ : clipping prevents pushing a bad action too far down in one step.
- Allows several minibatch updates from the same rollouts.

Reuse data, but limit how far each update moves the action distribution.

# Why Deep RL Is Hard

---

## Five Difficulties Read from the Trajectory Law

$$p_{\pi_\theta}(\tau) = \rho_0(s_0) \prod_{t=0}^{T-1} \pi_\theta(a_t \mid s_t) P(s_{t+1} \mid s_t, a_t).$$

Changing  $\theta$  changes predictions *and* the future data distribution.

| Difficulty            | Source in the math                          | Common response                   |
|-----------------------|---------------------------------------------|-----------------------------------|
| Correlated samples    | trajectory ordering                         | replay buffer                     |
| Distribution shift    | data sampled under changing $\pi_\theta$    | fresh evaluation rollouts         |
| Delayed/sparse reward | return $G_t$ spans time                     | value functions, shaping          |
| Moving targets        | $Y_t$ uses $V_\phi$ or $Q_{\tilde{\theta}}$ | target networks, smaller steps    |
| Exploration gap       | greedy misses unseen actions                | $\epsilon$ -greedy, entropy bonus |

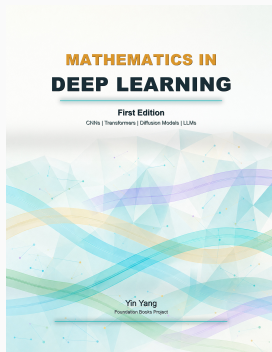
Practical diagnostics are checks on the same MDP, return, and Bellman objects.

# Summary

---

- **Setting:** state, action, reward, trajectory —  $S_t, A_t, R_{t+1}, S_{t+1}$ . The policy creates its own data.
- **MDP:**  $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$  with the Markov property.
- **Policy:**  $\pi(a | s)$ , often softmax of network logits.
- **Return & objective:**  $G_t = \sum_k \gamma^k R_{t+k+1}$ ,  $J(\pi) = \mathbb{E}[G_0]$ .
- **Values & Bellman:**  $V^\pi, Q^\pi$  satisfy one-step self-consistency; optimal versions use  $\max_a$ ;  $\mathcal{T}$  is a  $\gamma$ -contraction.
- **Sample targets:** Monte Carlo vs. TD; Q-learning + DQN stabilizers (replay, target network).
- **Policy gradients:** score-function identity, advantage as variance-reduced signal, actor-critic, PPO clipping.
- **Deep RL difficulty:** correlation, distribution shift, delayed reward, moving targets, exploration — all read from  $p_{\pi_\theta}(\tau)$ .

**Next: applying these objects to large-scale agents —  
preference-based training, language-model policies, and  
reward modeling.**



## Mathematics in Deep Learning

Chapter overview slides for the book by Yin Yang.

|           |                                                                                                                                                   |
|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| Book page | <a href="https://foundation-books.github.io/mathematics-in-deep-learning">foundation-books.github.io/mathematics-in-deep-learning</a>             |
| Code      | <a href="https://github.com/foundation-books/mathematics-in-deep-learning-code">github.com/foundation-books/mathematics-in-deep-learning-code</a> |
| Paperback | <a href="https://amazon.com/dp/BOH1ZZHDT2">amazon.com/dp/BOH1ZZHDT2</a>                                                                           |
| Hardcover | <a href="https://amazon.com/dp/BOH1ZL11MJ">amazon.com/dp/BOH1ZL11MJ</a>                                                                           |
| Kindle    | <a href="https://amazon.com/dp/B0GX32M8SP">amazon.com/dp/B0GX32M8SP</a>                                                                           |
| License   | CC BY-NC-ND 4.0                                                                                                                                   |

These free overview slides may be shared non-commercially with attribution and without modifications. Full explanations, exercises, and companion-code context are in the book and linked repository.